Project Report
On

eAuto – Bike Spare Parts E-Commerce Platform WFS

In partial fulfillment for the award of the degree Of
BACHELOR OF COMPUTER APPLICATION
[B.C.A]
Year 2025-2026

SUBMITTED BY:
Dhruv Dholiya (4924)
Dharmik Dobariya (4928)
Vatsal Kapadia (5029)

GUIDED BY:
Prof. Bipin Jadhav

BCA-5th SEMESTER

Submitted to:



SHRI SHAMBHUBHAI V. PATEL COLLEGE OF COMPUTER SCIENCE & BUSINESS

MANAGEMENT

Affiliated to

Veer Narmad South Gujarat University

SHRI SHAMBHUBHAI V. PATEL COLLEGE OF COMPUTER SCIENCE & BUSINESS MANAGEMENT

Near I. C. Gandhi School, Sumul Dairy Road, Surat (Gujarat), India

CERTIFICATE

This is to certify that the project entitled eAuto - Bike Spare Parts E-Commerce Platform *in Subject* Web Framework And Services *has been carried out by Mr. / Ms.* Dholiya Dhruv Bhareshbhai. *in Bachelor of Computer Application, Semester V, Exam Seat No.* 4924 *towards partial fulfillment of the course, for the Academic Year 2025-26.*

Shri Shambhubhai V. Patel College of Computer Science & Business Management * SURAT

Mentor/Guide
Date:

I/c Principal
Date:

PROJECT OF B.C.A.

Academic Year: 2025-26

Approved by: Dr. Nilesh Desai

Examiners

SHRI SHAMBHUBHAI V. PATEL COLLEGE OF COMPUTER SCIENCE & BUSINESS MANAGEMENT

Near I. C. Gandhi School, Sumul Dairy Road, Surat (Gujarat), India

CERTIFICATE

This is to certify that the project entitled Bike Spare Parts E-Commerce Platform *in Subject* Web Framework And Servies *has been carried out by Mr. / ~~Ms.~~* Dobariya Dharmik Sanjaybhai *in Bachelor of Computer Application, Semester V, Exam Seat No.* 4928 *towards partial fulfillment of the course, for the Academic Year 2025-26.*

Shri Shambhubhai V. Patel College of Computer Science & Business Management * SURAT

Mentor/Guide
Date:

I/c Principal
Date:

PROJECT OF B.C.A.

Academic Year: 2025-26

Approved by: Dr. Nidhi Desai

Examiners

SHRI SHAMBHUBHAI V. PATEL COLLEGE OF COMPUTER SCIENCE & BUSINESS MANAGEMENT

Near I. C. Gandhi School, Sumul Dairy Road, Surat (Gujarat), India

CERTIFICATE

This is to certify that the project entitled Bike Spare Parts E-commerce platform *in Subject* Web framework and services *has been carried out by Mr. / ~~Ms.~~* Kapadiya Vatsal Vipulkumar *in Bachelor of Computer Application, Semester V, Exam Seat No.* 5029 *towards partial fulfillment of the course, for the Academic Year 2025-26.*

Mentor/Guide
Date:

I/c Principal
Date:

PROJECT OF B.C.A.

Academic Year: 2025-26

Approved by: Dr. Nikunj Desai

Examiners

Acknowledgment

We would like to take this opportunity to express our heartfelt gratitude to everyone who contributed to the successful completion of our project, **"eAuto – Bike Spare Parts E-Commerce Platform."**

We are especially grateful to our project guide, **Prof. Bipin Jadhav**, for her invaluable guidance, continuous encouragement, and insightful feedback throughout the course of this work. Her expert advice and constant motivation have been a source of inspiration and have greatly helped us in achieving our objectives.

We also wish to extend our thanks to all the faculty members of our department for their cooperation, constructive suggestions, and for providing us with the necessary resources and knowledge that supported our project development.

Our sincere appreciation goes to our friends and classmates, whose timely help and support provided us with motivation and confidence during various phases of this project.

Lastly, we would like to acknowledge the dedication, teamwork, and mutual support of our project members, without whom this work would not have been possible:

- **Dhruv Dholiya (4924)**
- **Dharmik Dobariya (4928)**

- **Vatsal Kapadia (5029)**

Together, we have shared valuable experiences that will remain with us beyond the completion of this project.

Index

SR.NO	PARTICULAR	PAGE NO
1	**Introduction**	
	1.1 Background of the Project	
	1.2 Purpose of the Project	
	1.3 Scope of the Project	
	1.4 Objectives	
	1.5 Advantages of the System	
2	**Aim of the Project**	
	2.1 Statement of Aim	
	2.2 Importance of Aim	
	2.3 How Aim Solves Current Problems	
3	**Features of the Project**	
	3.1 User-side Features	
	3.1.1 User Registration & Login	
	3.1.2 Browse & Search Products	
	3.1.3 Cart & Checkout System	
	3.1.4 Order Tracking	
	3.1.5 Profile & Delivery Address Management	
	3.2 Admin-side Features	
	3.2.1 Admin Login & Dashboard	
	3.2.2 Manage Products (Add/Edit/Delete)	
	3.2.3 Manage Orders	
	3.2.4 Manage Users	
	3.2.5 Generate Reports	
4	**System Analysis & Design**	
	4.1 Requirement Gathering	
	4.2 Functional Requirements	
	4.3 Non-Functional Requirements	
	4.4 Technology Stack	
	4.5 Database Design	
5.	**E-R Diagram**	
	5.1 Entities	
	5.2 Relationships	
	5.3 Attributes	
	5.4 Final ER Model	
6.	**Data Flow Diagram (DFD)**	
	6.1 Context Level DFD (Level 0)	
	6.2 Level 1 DFD – User Processes	
	1.0 Login & Registration	

	2.0 Product Management	
	3.0 Cart & Order Management	
	4.0 Payment Processing	
	5.0 Supplier & Inventory Management	
	6.0 Delivery Management	
	7.0 Reports & Analytics	
	6.3 Explanation of Data Stores (MongoDB Collections)	
7	**Flow Chart**	
	7.1 User Registration & Login Flow	
	7.2 Product Purchase Flow	
	7.3 Order Management Flow	
	7.4 Payment Processing Flow	
8	**Implementation Details**	
	8.1 Frontend (React JS, Bootstrap, AOS)	
	8.2 Backend (PHP – Core PHP for APIs & Logic)	
	8.3 Database (MongoDB Compass – Collections & Schema)	
	8.4 Security Features (Authentication, Session Management, Validation)	
9	**Testing**	
	9.1 Test Cases (Unit & Integration)	
	9.2 User Testing (UI/UX & Functional)	
	9.3 Admin Testing	
	9.4 Performance & Load Testing	
10	**Limitations of the Project**	
	10.1 Technical Limitations (React-PHP integration, MongoDB structure)	
	10.2 Functional Limitations (Limited Payment Methods, Delivery System)	
	10.3 Future Enhancements (Mobile App, AI-based Recommendations, Advanced Reports)	
11	**Conclusion**	
	11.1 Summary of Work	
	11.2 Achievements	
	11.3 Future Scope	
12	**References**	

1. Introduction

1.1 Background of the Project

With the rapid growth of e-commerce, traditional methods of purchasing automobile spare parts are becoming outdated. Customers often face difficulties such as limited availability of parts, lack of price transparency, and difficulty in finding reliable vendors. The eAuto platform aims to address these issues by providing a centralized online marketplace for bike spare parts, offering convenience, affordability, and accessibility.

1.2 Purpose of the Project

The primary purpose of this project is to build an online platform where users can easily browse, compare, and purchase bike spare parts. The system benefits both customers (by offering a wide range of products and hassle-free shopping) and administrators (by providing tools to manage products, users, orders, and deliveries efficiently).

1.3 Scope of the Project

The scope of the eAuto project includes:

- User-side features such as product browsing, cart management, checkout, payment integration, order tracking, and user profile management.

- Admin-side features including product management, user management, order tracking, delivery assignment, and report generation.
- Database integration to maintain records of products, users, orders, and payments.
- Responsive front-end design for mobile and desktop users.

1.4 Objectives

The main objectives of the project are:

- To provide a user-friendly platform for purchasing bike spare parts online.
- To reduce the gap between suppliers and customers through a centralized system.
- To ensure secure payment processing and accurate order tracking.
- To enable administrators to manage inventory, users, and reports efficiently.
- To support future scalability by adding more brands, products, and features.

1.5 Advantages of the System

The proposed system offers several benefits, such as:

- **Convenience**: Customers can order parts anytime, anywhere.
- **Variety**: Wide range of spare parts from multiple brands in one place.

- **Transparency**: Clear product details, pricing, and discounts.
- **Efficiency**: Faster order processing and delivery management.
- **Data-driven**: Sales and user data help admins make better business decisions.
- **Scalability**: System can easily integrate with additional brands, suppliers, or services in the future.

---

2. Aim of the Project

2.1 Statement of Aim

The aim of the eAuto project is to design and develop a comprehensive **online e-commerce platform** for bike spare parts that allows users to easily search, compare, and purchase products, while providing administrators with effective tools for managing products, users, orders, and delivery processes.

2.2 Importance of Aim

The importance of this aim lies in addressing the major challenges faced in the traditional spare parts market:

- **Accessibility**: Customers often struggle to find genuine spare parts at local shops.
- **Time Saving**: Instead of physically visiting multiple stores, users can buy parts online with just a few clicks.
- **Reliability**: Ensures quality assurance through verified sellers and transparent pricing.
- **Business Growth**: Helps suppliers and administrators expand their reach to a larger customer base.
- **Technological Advancement**: Shifts the spare parts industry toward digital transformation, aligning with modern consumer habits.

2.3 How Aim Solves Current Problems

The aim of this project provides solutions to the existing problems in the bike spare parts domain by:

- **Centralized Platform**: Eliminates the need for customers to visit multiple vendors by offering all spare parts in one place.
- **Real-time Availability**: Provides up-to-date stock and pricing information.
- **Secure Transactions**: Integrates reliable payment gateways to ensure safe purchases.
- **Order Tracking**: Customers can track the status of their orders and deliveries online.
- **Efficient Management**: Admin panel allows streamlined management of inventory, users, and reports.
- **Customer Satisfaction**: Reduces hassle, improves transparency, and enhances the overall buying experience.

---

3. Features of the Project

3.1 User-side Features

3.1.1 User Registration & Login

- New users can create an account by registering with their email, phone number, and password.
- Existing users can log in using secure authentication.
- Sessions are maintained for user activities like browsing, cart, and checkout.

3.1.2 Browse & Search Products

- Users can browse spare parts based on **brand (Hero, Honda, Bajaj, etc.)**, categories, or price range.
- A **search bar** enables quick product lookup by name or part number.
- Product details such as image, description, price, discounts, and availability are displayed.

3.1.3 Cart & Checkout System

- Users can add multiple products to their **shopping cart**.
- Cart preview is available in the header for easy access.

- Checkout includes selection of **delivery address, payment method (Cash, Card, UPI, NetBanking)**, and order confirmation.

3.1.4 Order Tracking

- Customers can view their **order history** and the **current status** (Pending, Processed, Shipped, Delivered).
- Real-time updates ensure transparency in the delivery process.

3.1.5 Profile & Delivery Address Management

- Users can manage their **personal details, password, and contact information**.
- Multiple delivery addresses can be added, edited, or deleted for convenience.

3.2 Admin-side Features

3.2.1 Admin Login & Dashboard

- Secure admin login with authentication.
- Dashboard displays **key statistics** like total users, orders, sales, and product inventory status.

3.2.2 Manage Products (Add/Edit/Delete)

- Admins can **add new products** with details like name, brand, price, discount, stock quantity, and image.
- Existing products can be **edited or updated**.
- Outdated or unavailable products can be **deleted** from the system.

3.2.3 Manage Orders

- Admins can view all orders placed by users.
- Orders can be updated with statuses such as **Pending, Approved, Shipped, Delivered, or Cancelled**.
- Delivery staff can be assigned to orders for timely dispatch.

3.2.4 Manage Users

- Admins can view and manage registered users.
- Option to **block or remove users** in case of fraudulent activity.
- User data (profile, order history) can be accessed for support purposes.

3.2.5 Generate Reports

- Reports can be generated for **sales, revenue, product performance, and customer activity**.
- Helps administrators analyze business growth and make data-driven decisions.

---

4. System Analysis & Design

4.1 Requirement Gathering

Requirement gathering is done by analyzing the needs of customers and administrators. Sources include surveys, existing system analysis, and direct feedback.

- **User Requirements**:
    - Easy product browsing and searching.
    - Secure login and profile management.
    - Cart, checkout, and payment options.
    - Order tracking and history.
- **Admin Requirements**:
    - Ability to manage products, users, and orders.
    - Generate reports for sales and performance.
    - Dashboard for real-time insights.

4.2 Functional Requirements

Functional requirements describe what the system should do:

1. **User-side**
    - Register/Login/Logout functionality.
    - Browse products by category/brand.
    - Search products with filters.

- Add to cart, view cart, and checkout.
- Manage profile and delivery address.
- Track orders and view history.

2. **Admin-side**
    - Secure login for admins.
    - Manage product details (add, update, delete).
    - Manage users and their activities.
    - Manage orders (status updates, assign delivery).
    - Generate sales and inventory reports.

4.3 Non-Functional Requirements

These define system qualities rather than features:

- **Performance**: Fast loading times, responsive design, optimized queries.
- **Scalability**: Support for future expansion with more users and products.
- **Security**: Encrypted passwords, secure payment gateway, session management.
- **Usability**: Simple and intuitive UI for both users and admins.
- **Availability**: High uptime with reliable hosting.
- **Maintainability**: Clean code structure, modular design, and easy updates.

4.4 Technology Stack

Front-End

- **React.js** → Component-based architecture for building dynamic and reusable UI.
- **HTML5** → Structure of web pages.
- **CSS3** → Styling, animations, and responsive layouts.
- **JavaScript (ES6+)** → Client-side interactivity and logic.
- **Bootstrap 5** → Predefined responsive UI components.
- **AOS (Animate On Scroll)** → Smooth scroll-based animations.
- **PHP (for legacy UI/session handling)** → Used for rendering certain UI components and managing sessions (in earlier implementation).

Back-End

- **Option 1 (Earlier Stack – PHP)**
  - **Core PHP** → Server-side scripting and session handling.
  - **XAMPP (Apache + PHP + MySQL)** → Local development server.
- **Option 2 (Modern Stack – Node.js)**
  - **Node.js with Express.js** → REST API development and backend logic.
  - **Integration with MySQL** → For managing product, user, and order data.

Database

- **MySQL** → Primary relational database for structured data (Users, Products, Orders, Cart, Reports).

- **MongoDB Compass (Optional)** → GUI tool for managing MongoDB if a hybrid/noSQL database is considered for future scalability.

Tools & Platforms

- **XAMPP / Node Server** → For local testing and development.
- **Git & GitHub** → Version control and collaborative development.
- **MongoDB Compass** → Database GUI for NoSQL database management (if used).

Payment Integration

- **Razorpay / PayPal / Stripe APIs** → Secure and seamless online payments via multiple modes (Cards, UPI, NetBanking, Wallets).

4.5 Database Design

1. Admins Table

Used for admin login and management.

```
CREATE TABLE admins (
   id INT AUTO_INCREMENT PRIMARY KEY,
   email VARCHAR(100) NOT NULL UNIQUE,
   password VARCHAR(255) NOT NULL,
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Fields

- **id**: Unique identifier (Primary Key).
- **email**: Admin login email (unique).
- **password**: Hashed password for secure login.
- **created_at**: Timestamp when admin account was created.

---

2. Users Table

Stores customer account details.

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    fullname VARCHAR(255),
    email VARCHAR(255) UNIQUE,
    contact VARCHAR(20),
    birthdate DATE,
    gender VARCHAR(10),
    password VARCHAR(255)
);
```

Fields

- **id**: User ID (Primary Key).

- **fullname**: User’s full name.
- **email**: Unique email ID.
- **contact**: Mobile number.
- **birthdate**: Date of birth.
- **gender**: Gender (Male/Female/Other).
- **password**: Hashed password for login.

---

3. Cart Table

Maintains items added to the cart by the user.

```
CREATE TABLE cart (
    id INT AUTO_INCREMENT PRIMARY KEY,
    product_name VARCHAR(255) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    image VARCHAR(255),
    brand VARCHAR(100),
    quantity INT DEFAULT 1
);
```

Fields

- **id**: Cart ID (Primary Key).

- **product_name**: Name of the product.
- **price**: Current price.
- **image**: Product image path.
- **brand**: Brand name.
- **quantity**: Quantity selected.

---

4. Orders Table

Stores order-level details.

```
CREATE TABLE orders (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  fullname VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL,
  phone VARCHAR(20) NOT NULL,
  address TEXT NOT NULL,
  city VARCHAR(100) NOT NULL,
  state VARCHAR(100) NOT NULL,
  zip VARCHAR(20) NOT NULL,
  total_amount DECIMAL(10,2) NOT NULL,
  payment_method VARCHAR(50) NOT NULL,
```

```
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Fields

- **id**: Order ID (Primary Key).
- **user_id**: Customer who placed the order (FK to users).
- **fullname, email, phone**: Customer details.
- **address, city, state, zip**: Shipping address.
- **total_amount**: Total payable amount.
- **payment_method**: (COD, UPI, Card, etc.).
- **created_at**: Timestamp when the order was placed.

---

5. Order Items Table

Stores individual product details inside each order.

```
CREATE TABLE order_items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  order_id INT NOT NULL,
  product_id INT NOT NULL,
  product_name VARCHAR(255) NOT NULL,
```

```
  quantity INT NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
  FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE
);
```

Fields

- **id**: Order Item ID (Primary Key).
- **order_id**: Linked to orders.id (FK).
- **product_id**: Linked to products.id (FK).
- **product_name**: Product name.
- **quantity**: Quantity purchased.
- **price**: Price at purchase time.

---

6. Products Table

Stores details of all available products.

```
CREATE TABLE products (
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    product_name VARCHAR(255) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    original_price DECIMAL(10,2),
    discount_amount DECIMAL(10,2),
    image VARCHAR(255),
    brand VARCHAR(100),
    main_brand ENUM('BRAND B', 'Hero', 'Honda') NOT NULL,
    rating FLOAT,
    reviews INT,
    stock_status ENUM('In Stock', 'Sold Out') DEFAULT 'In Stock'
);
```

Fields

- **id**: Product ID (Primary Key).
- **product_name**: Product title.
- **price**: Selling price.
- **original_price**: Price before discount.
- **discount_amount**: Discount value applied.
- **image**: Product image path.
- **brand**: Sub-brand name.
- **main_brand**: Main brand category (Hero, Honda, Brand B).
- **rating**: Average product rating.

- **reviews**: Number of reviews.
- **stock_status**: Availability (In Stock / Sold Out).

◆ **Database Summary**

- **Admins**: Manage platform access.
- **Users**: Store customer info.
- **Products**: Store inventory data.
- **Cart**: Temporary cart items.
- **Orders + Order Items**: Complete order tracking.

---

5. E–R Diagram

5.1 Entities

From the database design, the main entities are:

1. **Admins**
2. **Users**
3. **Products**
4. **Cart**
5. **Orders**
6. **Order_Items**
7. **Delivery Address (inside Orders table)**

5.2 Relationships

1. **User → Orders**:
    - One user can place many orders.
    - Relationship: **1:N**
2. **Orders → Order_Items**:
    - One order can contain many products (items).
    - Relationship: **1:N**
3. **Products → Order_Items**:
    - One product can appear in many orders.
    - Relationship: **1:N**

4. **User → Cart**:
    - One user can maintain many cart items.
    - Relationship: **1:N**
5. **Products → Cart**:
    - A product can exist in many users' carts.
    - Relationship: **1:N**
6. **Admins → Products/Orders/Users**:
    - Admins manage products, orders, and users.
    - Relationship: **1:N (management)**

5.3 Attributes

Admins

- id (PK), email, password, created_at

Users

- id (PK), fullname, email, contact, birthdate, gender, password

Products

- id (PK), product_name, price, original_price, discount_amount, image, brand, main_brand, rating, reviews, stock_status

Cart

- id (PK), product_name, price, image, brand, quantity

Orders

- id (PK), user_id (FK), fullname, email, phone, address, city, state, zip, total_amount, payment_method, created_at

Order_Items

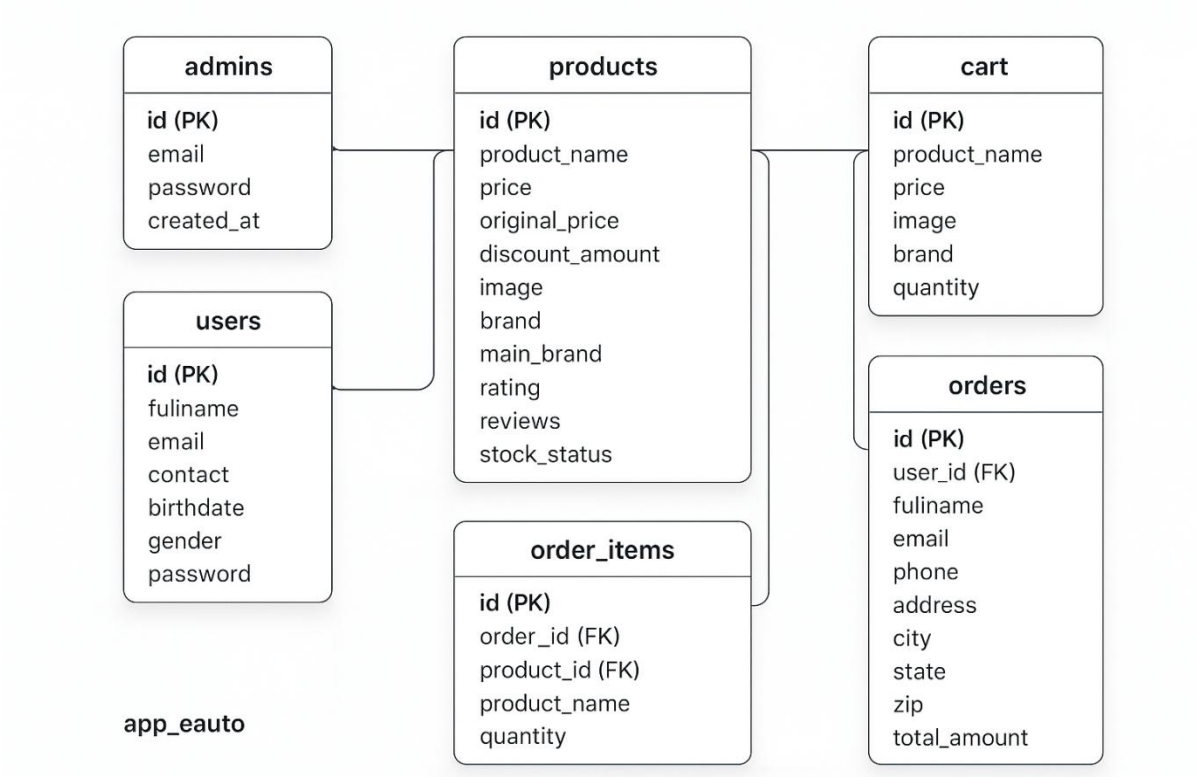
- id (PK), order_id (FK), product_id (FK), product_name, quantity, price

---

5.4 Final ER Model

The final ER Model will include:

- **Users** linked to **Cart** and **Orders**
- **Orders** linked to **Order_Items**
- **Products** linked to **Cart** and **Order_Items**
- **Admins** linked to **Products, Orders, Users**



---

6. Data Flow Diagram (DFD)

6.1 Context Level DFD (Level 0)

Central Process

eAuto Management System

High-Level Sub-Processes

1. **Login & Registration** – Handles Customer/Admin authentication
2. **Product Management** – Search, browse, and manage product catalog
3. **Cart & Order Management** – Add to cart, checkout, confirm orders
4. **Payment Processing** – Verify and confirm transactions
5. **Supplier & Inventory Management** – Stock updates, stock requests
6. **Delivery Management** – Assign deliveries, track status
7. **Reports & Analytics** – Generate sales, stock, and delivery reports for Admin

---

External Entities

- **Customer**
- **Admin**
- **Supplier**
- **Delivery Staff**

Data Stores

Data Store	Description
D1: User Database	Customer & admin login/registration details
D2: Product Database	Product details, categories, pricing, discounts
D3: Cart & Order Database	Cart info, order details, order history
D4: Payment/Transaction Database	Payment confirmations, transaction logs
D5: Inventory Database	Stock levels, supplier updates
D6: Delivery Database	Delivery assignments, status updates
D7: Reports Database	Aggregated data for admin reports/analytics

---

Data Flow (Movement)

Customer ↔ eAuto System

- **Requests:** Register/Login, Search Products, Add to Cart, Place Order, Payment, View Orders
- **Responses:** Login Confirmation, Search Results, Order Confirmation, Payment Status, Delivery Status
- **Databases Accessed:** D1, D2, D3, D4, D6

Admin ↔ eAuto System

- **Requests:** Manage Products, Orders, Users; View Reports
- **Responses:** Confirmation Messages, Updated Data, Reports
- **Databases Accessed:** D1, D2, D3, D5, D6, D7

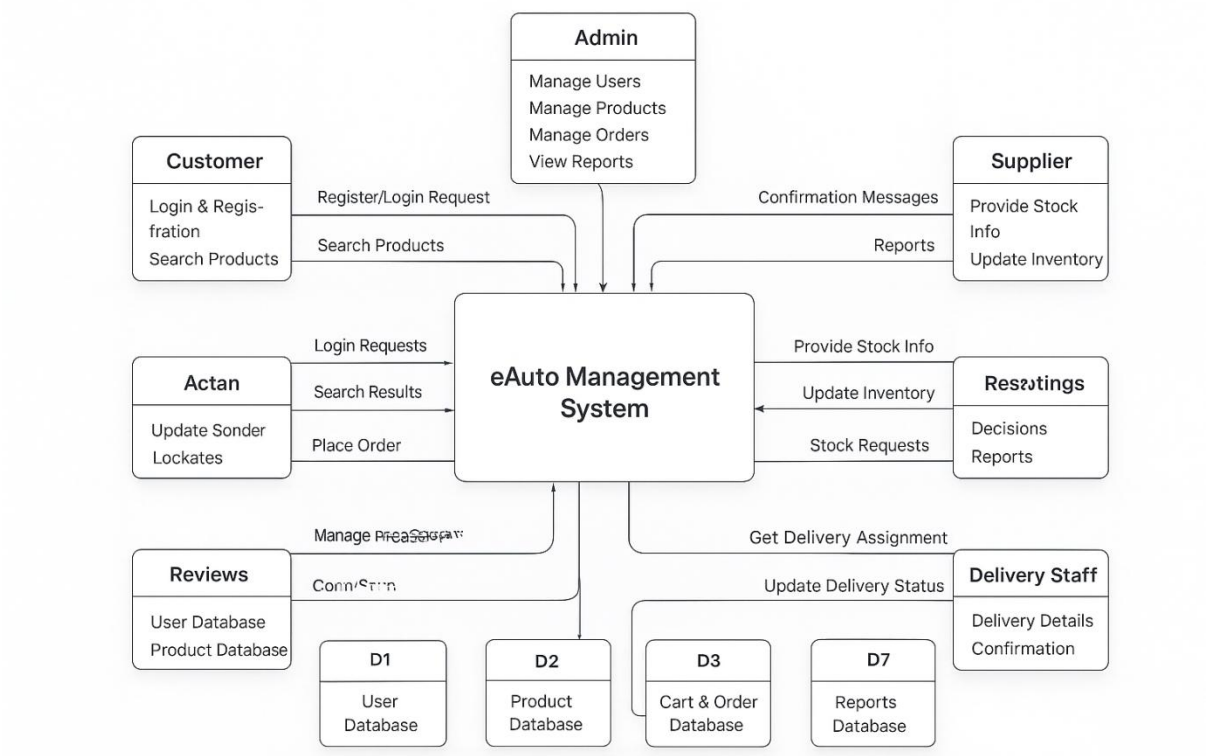
Supplier ↔ eAuto System

- **Requests:** Provide Stock Info, Update Inventory

- **Responses:** Stock Confirmation, Order Requests
- **Databases Accessed:** D5

Delivery Staff ↔ eAuto System

- **Requests:** Delivery Assignment, Update Delivery Status
- **Responses:** Delivery Details, Confirmation
- **Databases Accessed:** D3, D6



6.2 Level 1 DFD – User Processes

Processes (Inside the System)

Process	Input	Output	Data Store
1.0 Login & Registration	Login/Register request (Customer/Admin)	Login confirmation	D1: User Database
2.0 Product Management	Product search request (Customer), Add/Update product (Admin)	Product info / Search results	D2: Product Database
3.0 Cart & Order Management	Add to Cart, Place Order (Customer)	Order confirmation	D3: Cart & Order Database
4.0 Payment Processing	Payment details (Customer)	Transaction confirmation	D4: Payment Records
5.0 Supplier & Inventory Management	Stock info update (Supplier)	Stock confirmation	D5: Inventory Database
6.0 Delivery Management	Delivery assignment request (System → Delivery Staff)	Delivery confirmation, Status update	D6: Delivery Records
7.0 Reports & Analytics	Report request (Admin)	Sales / Stock / Delivery reports	D7: Reports Database

---

External Entities

- **Customer** – Register/Login, Search Products, Add to Cart, Make Payment, Track Delivery
- **Admin** – Manage Products, Orders, Users, Reports
- **Supplier** – Stock Updates, Inventory Info
- **Delivery Staff** – Order Delivery, Update Delivery Status

---

Data Stores

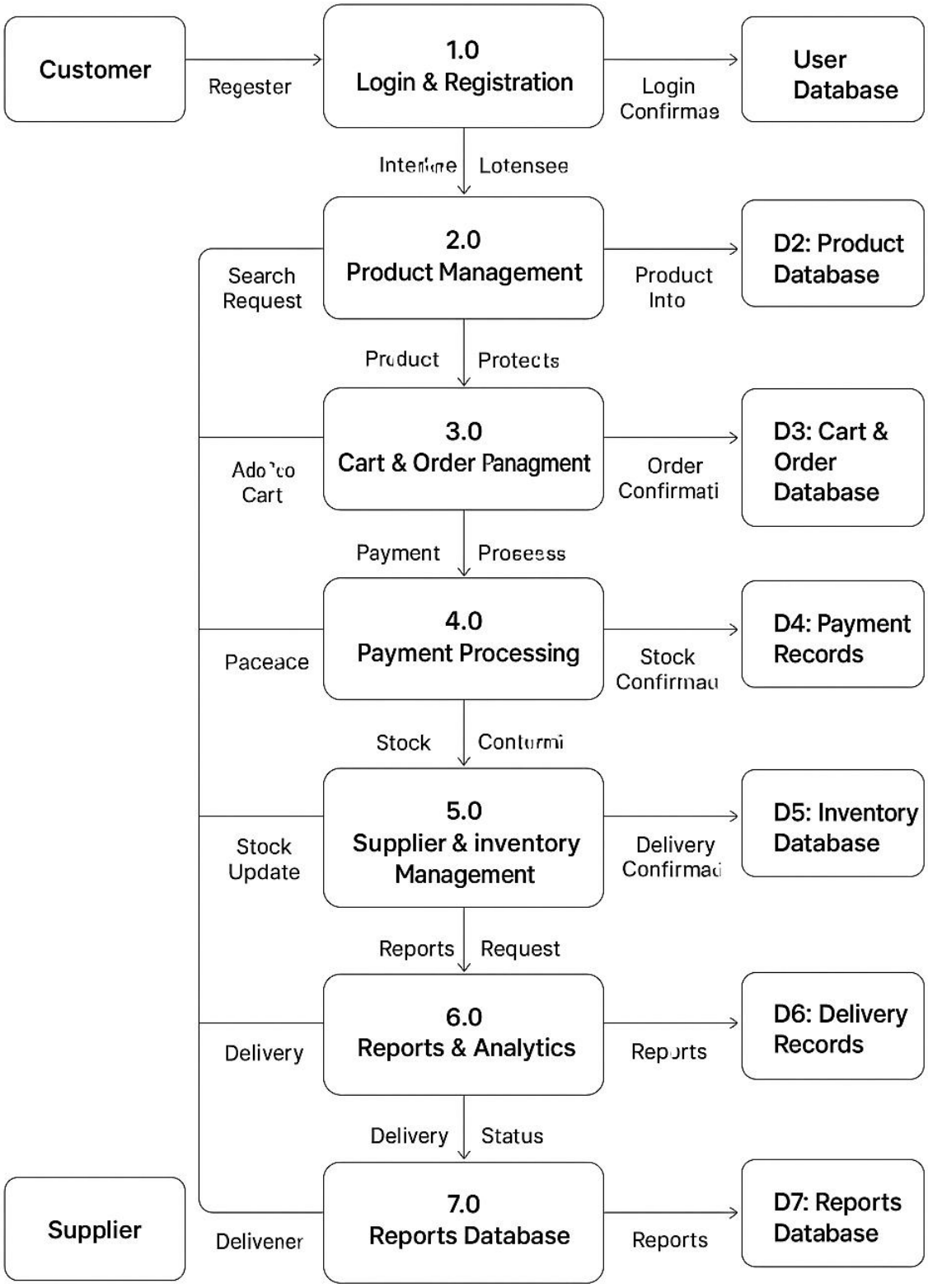
Data Store	Description
D1: User Database	Stores customer/admin login info
D2: Product Database	Stores product details
D3: Cart & Order Database	Stores cart & order transactions
D4: Payment Records	Stores payment details
D5: Inventory Database	Manages stock levels
D6: Delivery Records	Stores delivery status and assignments
D7: Reports Database	Stores analytics data

Data Flows

Entity	Process	Data Store
Customer ↔ 1.0 Login & Registration ↔ D1	Authentication / Registration	User Database
Customer ↔ 2.0 Product Management ↔ D2	Product search / browsing	Product Database
Customer ↔ 3.0 Cart & Order Management ↔ D3	Add to cart, Place order	Cart & Order Database
Customer ↔ 4.0 Payment Processing ↔ D4	Payment processing	Payment Records
Supplier ↔ 5.0 Supplier & Inventory Management ↔ D5	Stock updates	Inventory Database
Delivery Staff ↔ 6.0 Delivery Management ↔ D6	Delivery assignments / status updates	Delivery Records
Admin ↔ 7.0 Reports & Analytics ↔ D7	Generate reports	Reports Database

---

eAuto Management System – Level 1 DFD



1.0 Login & Registration

External Entities

- Customer
- Admin

Data Store

- D1: User Database

Sub-Processes

1.1 Register User

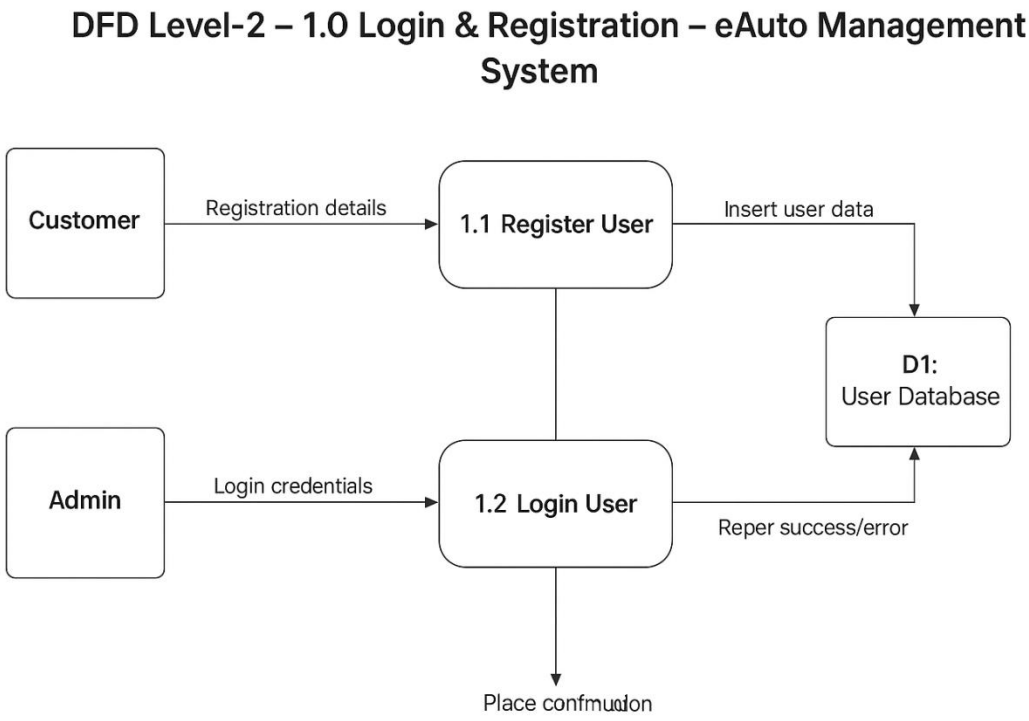
- **Input:** Registration details (name, email, password, etc.) from Customer/Admin
- **Process:** Validate details, store new record
- **Output:** Registration confirmation
- **Data Store Interaction:** Insert new user into **D1: User Database**

1.2 Login User

- **Input:** Login credentials (username/email & password) from Customer/Admin
- **Process:** Validate credentials
- **Output:** Login success/failure message
- **Data Store Interaction:** Verify credentials from **D1: User Database**

Data Flows

From	To	Data
Customer/Admin → 1.1 Register User	Registration details	Input
1.1 Register User → D1: User Database	Store data	Insert
Customer/Admin → 1.2 Login User	Login credentials	Input
1.2 Login User → D1: User Database	Verify credentials	Query
1.1 Register User → Customer/Admin	Registration confirmation	Output
1.2 Login User → Customer/Admin	Login confirmation / error	Output



2.0 Product Management

External Entities

- Customer
- Admin

Data Store

- D2: Product Database

Sub-Processes

2.1 Search Product

- **Input:** Product search request from Customer
- **Process:** Retrieve matching product info
- **Output:** Search results (product list/details)
- **Data Store Interaction:** Read from **D2: Product Database**

2.2 Add Product

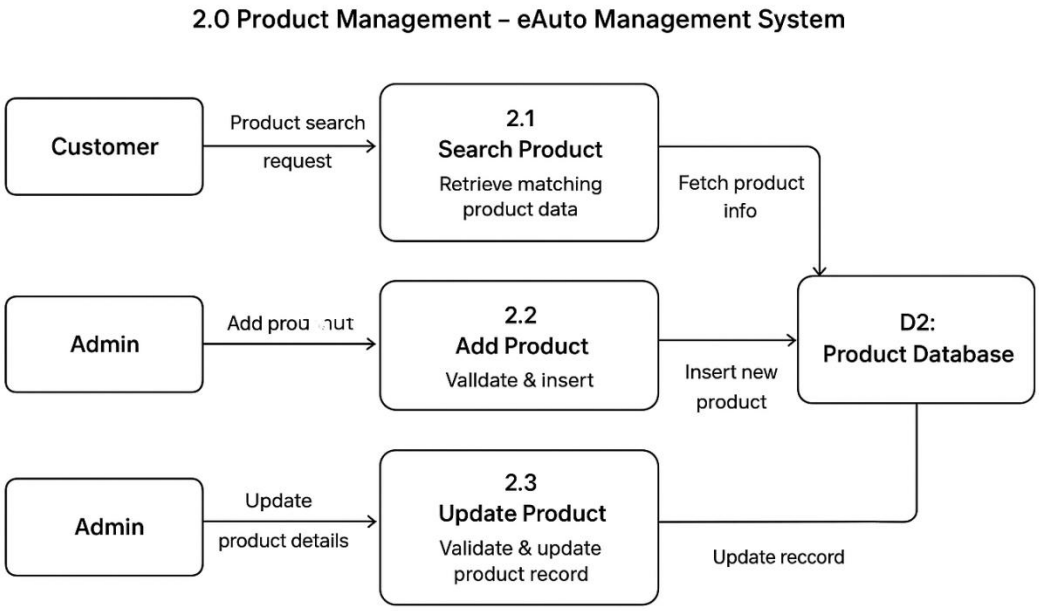
- **Input:** New product details from Admin
- **Process:** Validate and insert new product
- **Output:** Product added confirmation
- **Data Store Interaction:** Insert into **D2: Product Database**

2.3 Update Product

- **Input:** Updated product details from Admin
- **Process:** Validate and update product record
- **Output:** Update confirmation
- **Data Store Interaction:** Update record in **D2: Product Database**

Data Flows

From	To	Data
Customer → 2.1 Search Product	Product search request	Input
2.1 Search Product → D2: Product Database	Fetch product info	Query
D2: Product Database → 2.1 Search Product	Product details	Response
2.1 Search Product → Customer	Search results	Output
Admin → 2.2 Add Product	New product details	Input
2.2 Add Product → D2: Product Database	Insert new product	Insert
2.2 Add Product → Admin	Add confirmation	Output
Admin → 2.3 Update Product	Updated product details	Input
2.3 Update Product → D2: Product Database	Update record	Update
2.3 Update Product → Admin	Update confirmation	Output



3.0 Cart & Order Management

External Entities

- Customer

Data Store

- D3: Cart & Order Database

Sub-Processes

3.1 Add to Cart

- **Input:** Product selection from Customer
- **Process:** Validate product availability and add to cart
- **Output:** Cart update confirmation
- **Data Store Interaction:** Insert/Update cart record in **D3: Cart & Order Database**

3.2 View Cart

- **Input:** Cart view request from Customer
- **Process:** Retrieve all items from cart
- **Output:** Cart details (products, quantity, total price)
- **Data Store Interaction:** Read from **D3: Cart & Order Database**

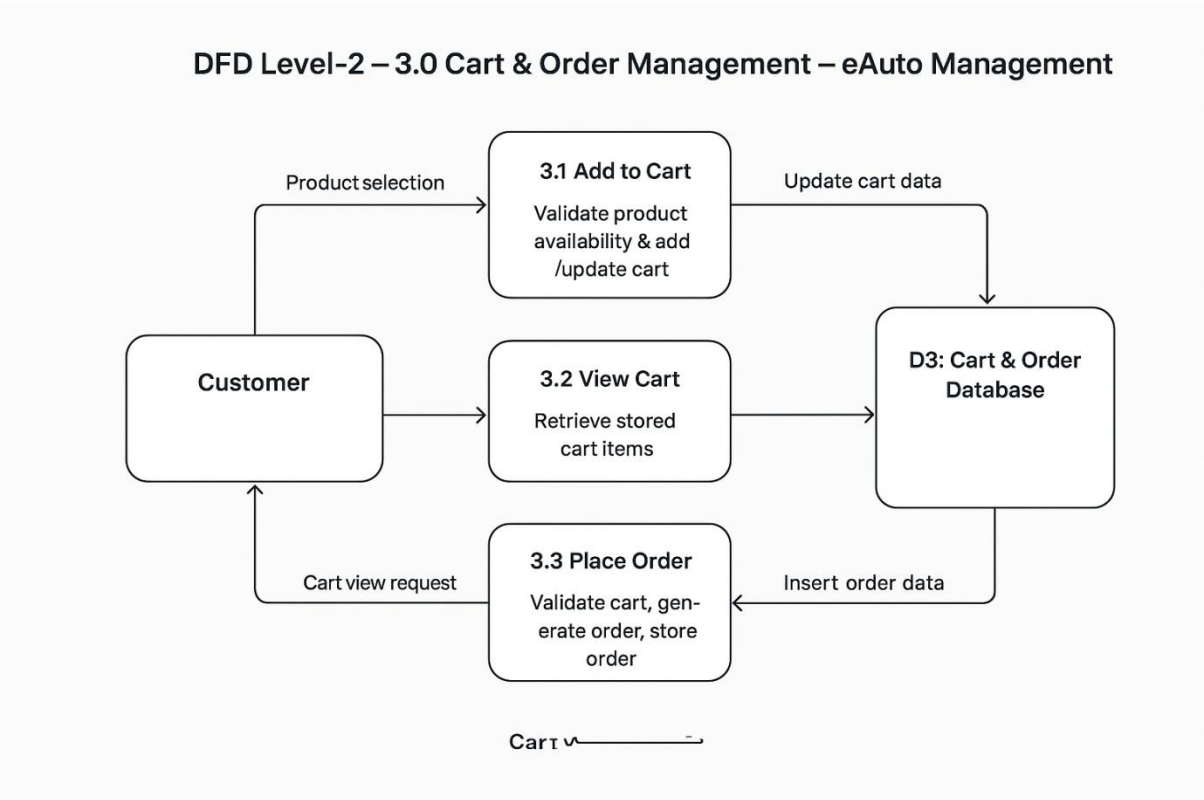
3.3 Place Order

- **Input:** Order request (checkout) from Customer
- **Process:** Validate cart items, generate order ID, store order
- **Output:** Order confirmation

- **Data Store Interaction:** Insert order details into **D3: Cart & Order Database**

Data Flows

From	To	Data
Customer → 3.1 Add to Cart	Product selection	Input
3.1 Add to Cart → D3: Cart & Order Database	Update cart data	Insert/Update
3.1 Add to Cart → Customer	Cart update confirmation	Output
Customer → 3.2 View Cart	Cart view request	Input
3.2 View Cart → D3: Cart & Order Database	Fetch cart items	Query
3.2 View Cart → Customer	Cart details	Output
Customer → 3.3 Place Order	Place order request	Input
3.3 Place Order → D3: Cart & Order Database	Insert order data	Insert
3.3 Place Order → Customer	Order confirmation	Output



4.0 Payment Processing

External Entities

- Customer
- Bank / UPI Gateway

Data Store

- D4: Payment Records

Sub-Processes

4.1 Initiate Payment

- **Input:** Payment details (card/UPI/net banking/cash request) from Customer
- **Process:** Validate input format and forward request
- **Output:** Payment initiation status
- **Data Flow:** Send payment details to Bank/UPI Gateway

4.2 Verify Payment Details

- **Input:** Payment validation request from 4.1
- **Process:** Bank/UPI Gateway checks account/card details
- **Output:** Valid/Invalid response
- **Data Flow:** Response sent back to 4.2 Verify Payment Details

4.3 Confirm Transaction

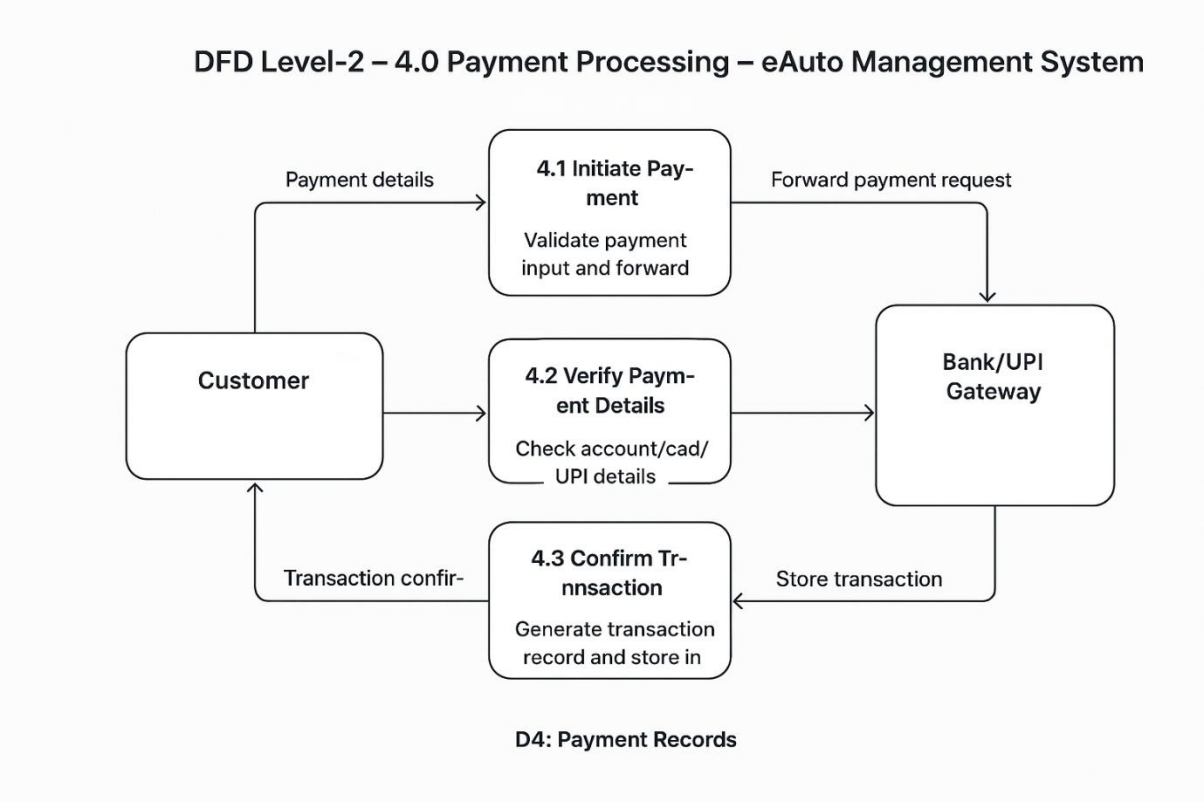
- **Input:** Valid payment confirmation from Bank/UPI Gateway
- **Process:** Generate transaction record and store in DB
- **Output:** Transaction confirmation / Failure message

- **Data Store Interaction:** Insert into **D4: Payment Records**

Data Flows

From	To	Data
Customer → 4.1 Initiate Payment	Payment details	Input
4.1 Initiate Payment → Bank/UPI Gateway	Forward payment request	Request
Bank/UPI Gateway → 4.2 Verify Payment Details	Verification response	Response
4.2 Verify Payment Details → 4.3 Confirm Transaction	Validated result	Input
4.3 Confirm Transaction → D4: Payment Records	Store transaction	Insert
4.3 Confirm Transaction → Customer	Transaction confirmation	Output

---



5.0 Supplier & Inventory Management

External Entities

- Supplier
- Admin

Data Store

- D5: Inventory Database

Sub-Processes

5.1 Update Stock

- **Input:** Stock info update from Supplier/Admin
- **Process:** Validate stock details and update inventory records
- **Output:** Stock update confirmation
- **Data Store Interaction:** Update **D5: Inventory Database**

5.2 Check Inventory

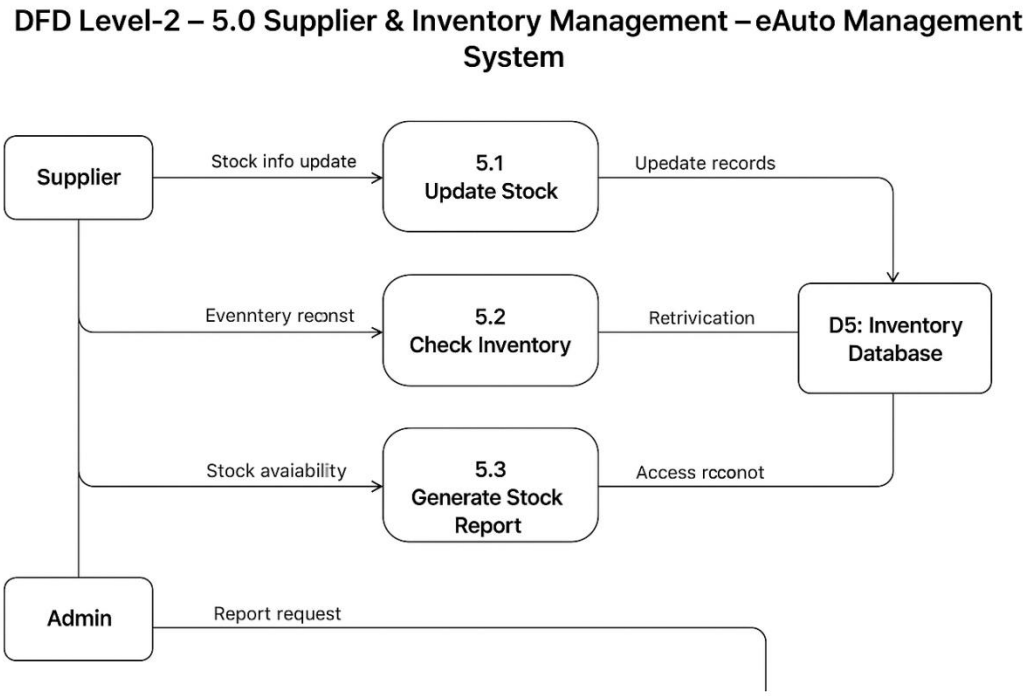
- **Input:** Inventory check request from Admin/System
- **Process:** Retrieve current stock details
- **Output:** Stock availability report
- **Data Store Interaction:** Read from **D5: Inventory Database**

5.3 Generate Stock Report

- **Input:** Report request from Admin
- **Process:** Compile stock levels, shortages, and supplier info
- **Output:** Inventory report
- **Data Store Interaction:** Access data from **D5: Inventory Database**

Data Flows

From	To	Data
Supplier/Admin → 5.1 Update Stock	Stock info update	Input
5.1 Update Stock → D5: Inventory Database	Update records	Update
5.1 Update Stock → Supplier/Admin	Stock confirmation	Output
Admin/System → 5.2 Check Inventory	Inventory check request	Input
5.2 Check Inventory → D5: Inventory Database	Retrieve stock data	Query
5.2 Check Inventory → Admin/System	Stock availability report	Output
Admin → 5.3 Generate Stock Report	Report request	Input
5.3 Generate Stock Report → D5: Inventory Database	Access data	Query
5.3 Generate Stock Report → Admin	Inventory report	Output



6.0 Delivery Management

External Entities

- System
- Delivery Staff
- Customer

Data Store

- D6: Delivery Records

Sub-Processes

6.1 Assign Delivery

- **Input:** Delivery assignment request from System
- **Process:** Assign order to available Delivery Staff
- **Output:** Delivery assignment confirmation
- **Data Store Interaction:** Insert assignment into **D6: Delivery Records**

6.2 Update Delivery Status

- **Input:** Delivery progress/status from Delivery Staff
- **Process:** Record status updates (Out for delivery, Delivered, Failed)
- **Output:** Updated delivery status
- **Data Store Interaction:** Update **D6: Delivery Records**

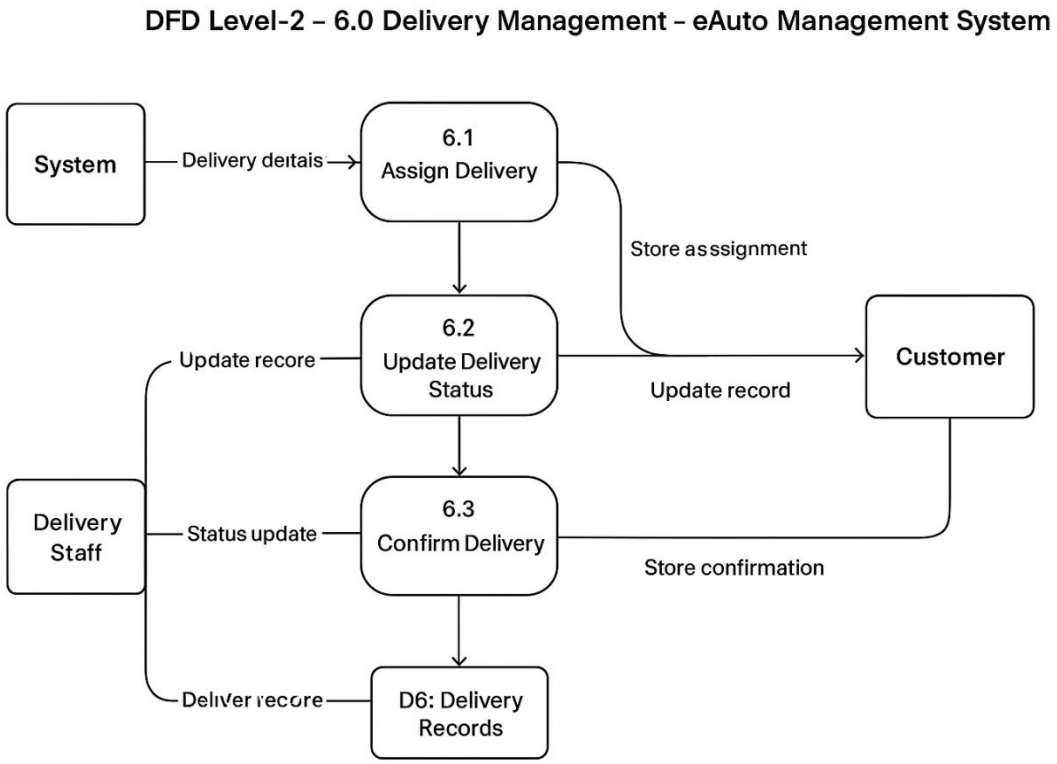
6.3 Confirm Delivery

- **Input:** Delivery completion info from Delivery Staff
- **Process:** Validate and confirm successful delivery
- **Output:** Delivery confirmation sent to Customer & System

- **Data Store Interaction:** Store confirmation in **D6: Delivery Records**

Data Flows

From	To	Data
System → 6.1 Assign Delivery	Delivery assignment request	Input
6.1 Assign Delivery → D6: Delivery Records	Store assignment	Insert
6.1 Assign Delivery → Delivery Staff	Assignment confirmation	Output
Delivery Staff → 6.2 Update Delivery Status	Status update	Input
6.2 Update Delivery Status → D6: Delivery Records	Update record	Update
6.2 Update Delivery Status → System/Customer	Updated status	Output
Delivery Staff → 6.3 Confirm Delivery	Delivery completion info	Input
6.3 Confirm Delivery → D6: Delivery Records	Store confirmation	Insert
6.3 Confirm Delivery → Customer	Delivery confirmation	Output
6.3 Confirm Delivery → System	Delivery confirmation	Output



7.0 Reports & Analytics

External Entities

- Admin
- System

Data Store

- D7: Reports Database

Sub-Processes

7.1 Generate Sales Report

- **Input:** Report request (sales) from Admin
- **Process:** Compile order and payment data to generate sales insights
- **Output:** Sales report
- **Data Store Interaction:** Retrieve/store in **D7: Reports Database**

7.2 Generate Stock Report

- **Input:** Report request (stock) from Admin
- **Process:** Collect inventory data, show shortages and availability
- **Output:** Stock report
- **Data Store Interaction:** Retrieve/store in **D7: Reports Database**

7.3 Generate Delivery Report

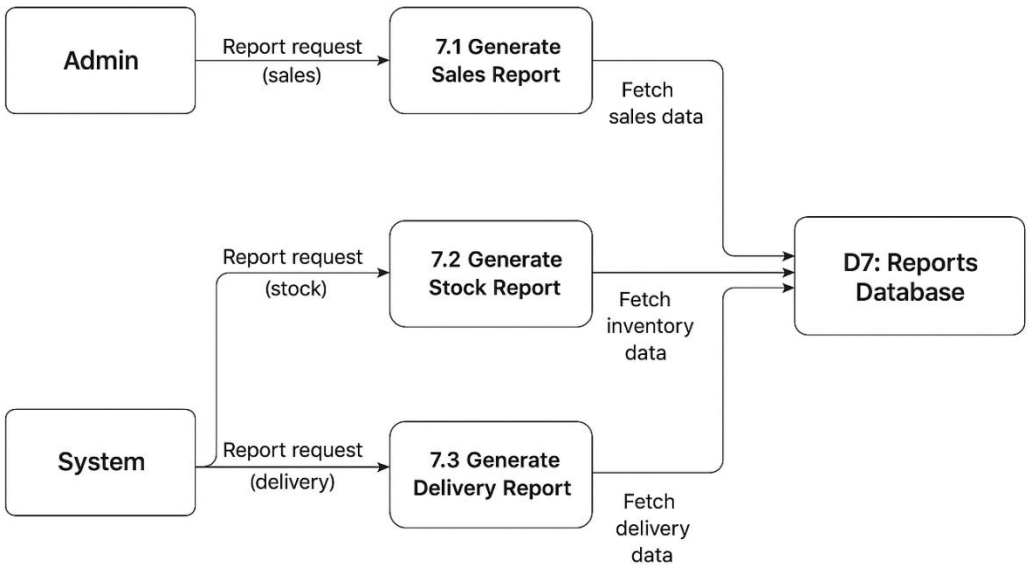
- **Input:** Report request (delivery) from Admin
- **Process:** Summarize delivery records, delays, completion rates
- **Output:** Delivery report
- **Data Store Interaction:** Retrieve/store in **D7: Reports Database**

Data Flows

From	To	Data
Admin → 7.1 / 7.2 / 7.3	Report request	Input
7.1 Generate Sales Report → D7: Reports Database	Fetch sales data	Query/Store
7.1 Generate Sales Report → Admin	Sales report	Output
7.2 Generate Stock Report → D7: Reports Database	Fetch inventory data	Query/Store
7.2 Generate Stock Report → Admin	Stock report	Output
7.3 Generate Delivery Report → D7: Reports Database	Fetch delivery data	Query/Store
7.3 Generate Delivery Report → Admin	Delivery report	Output
System → D7: Reports Database	Data sync (orders, inventory, delivery)	Update

---

DFD Level-2 – 7.0 Reports & Analytics – eAuto Management System



6.3 Explanation of Data Stores (MySQL)

D1: Users Table (Customers + Admins)

```
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    fullname VARCHAR(255) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    contact VARCHAR(20),
    role ENUM('customer', 'admin') DEFAULT 'customer'
);
```

- Stores both customers and admins.
- role differentiates them.

D2: Products Table

```
CREATE TABLE products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    brand VARCHAR(100),
    category VARCHAR(100),
    price DECIMAL(10,2) NOT NULL,
    stock INT DEFAULT 0,
    discount DECIMAL(5,2) DEFAULT 0,
```

```
    description TEXT,
    image_url VARCHAR(255)
);
```

- Product catalog with discount & stock count.

D3: Cart Table

(Each row = one product in a user's cart)

```
CREATE TABLE cart (
    cart_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT DEFAULT 1,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES products(product_id) ON DELETE CASCADE
);
```

- A user can have multiple cart items.

D4: Orders Table

```
CREATE TABLE orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    status ENUM('Pending','Shipped','Delivered','Cancelled') DEFAULT 'Pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

Order Items (child table for products inside an order)

```
CREATE TABLE order_items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

D5: Payments Table

```
CREATE TABLE payments (
    payment_id VARCHAR(50) PRIMARY KEY,
    order_id INT NOT NULL,
    method ENUM('UPI','Card','NetBanking','Wallet','COD') NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    status ENUM('Success','Failed','Pending') DEFAULT 'Pending',
    transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE
);
```

D6: Delivery Records Table

```
CREATE TABLE deliveries (
    delivery_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    delivery_staff VARCHAR(100),
    delivery_address TEXT NOT NULL,
    status ENUM('Assigned','Out for Delivery','Delivered') DEFAULT 'Assigned',
    delivered_at TIMESTAMP NULL,
```

```
    FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE
CASCADE
);
```

D7: Reports Table

```
CREATE TABLE reports (
    report_id INT AUTO_INCREMENT PRIMARY KEY,
    type ENUM('Sales','Stock','Customer','Delivery') NOT NULL,
    generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_sales DECIMAL(12,2),
    top_products TEXT   -- Can store JSON/CSV list of product names
);
```

---

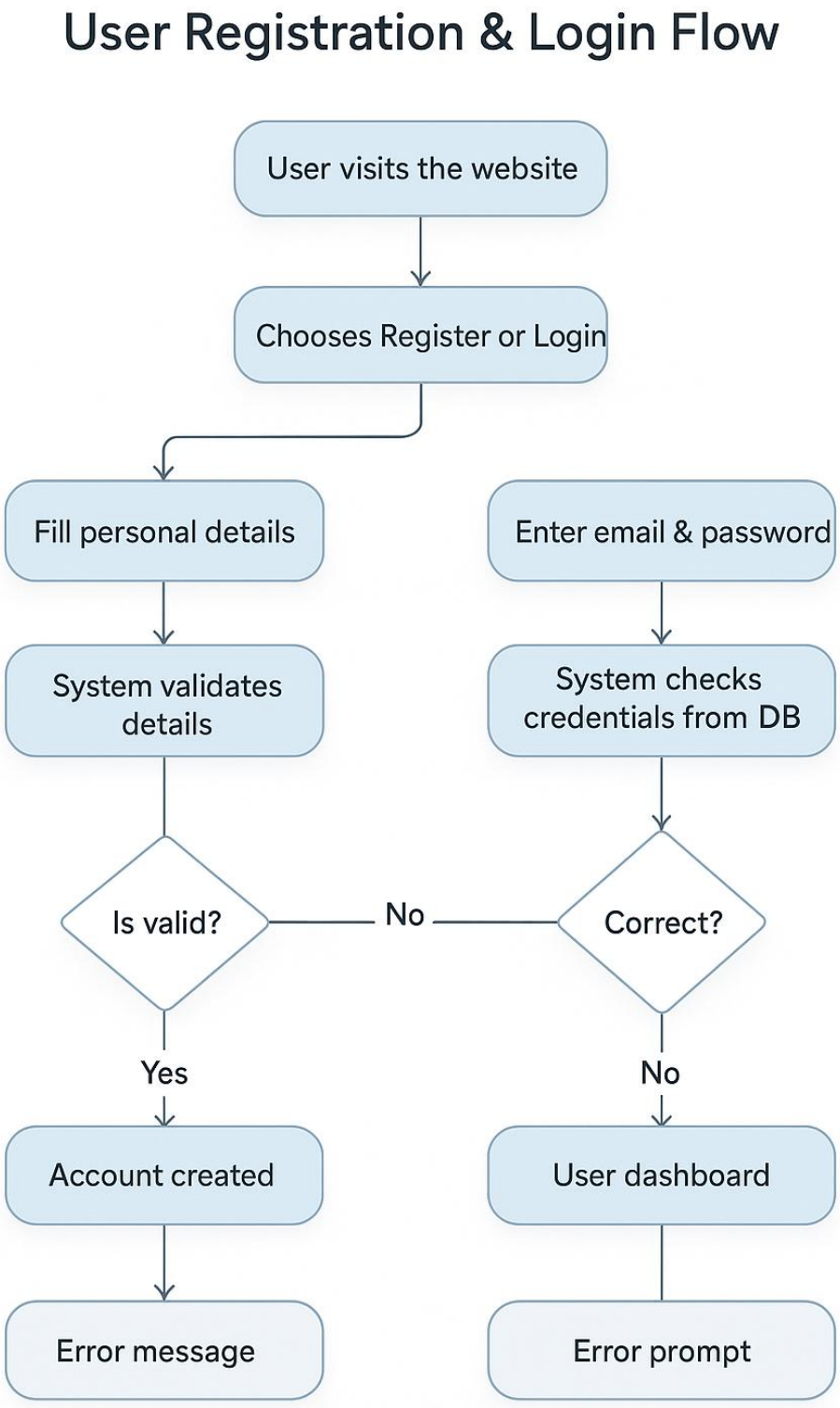
7. Flow Chart

7.1 User Registration & Login Flow

Process Steps:

1. User visits the website.
2. Chooses **Register** or **Login** option.
3. For **Registration**:
    - Fill personal details (name, email, password, contact).
    - System validates details.
    - If valid → Account created → Confirmation message.
    - If invalid → Error message.
4. For **Login**:
    - Enter email & password.
    - System checks credentials from DB.
    - If correct → User dashboard.
    - If incorrect → Error prompt.

Flow Chart Structure (Textual)

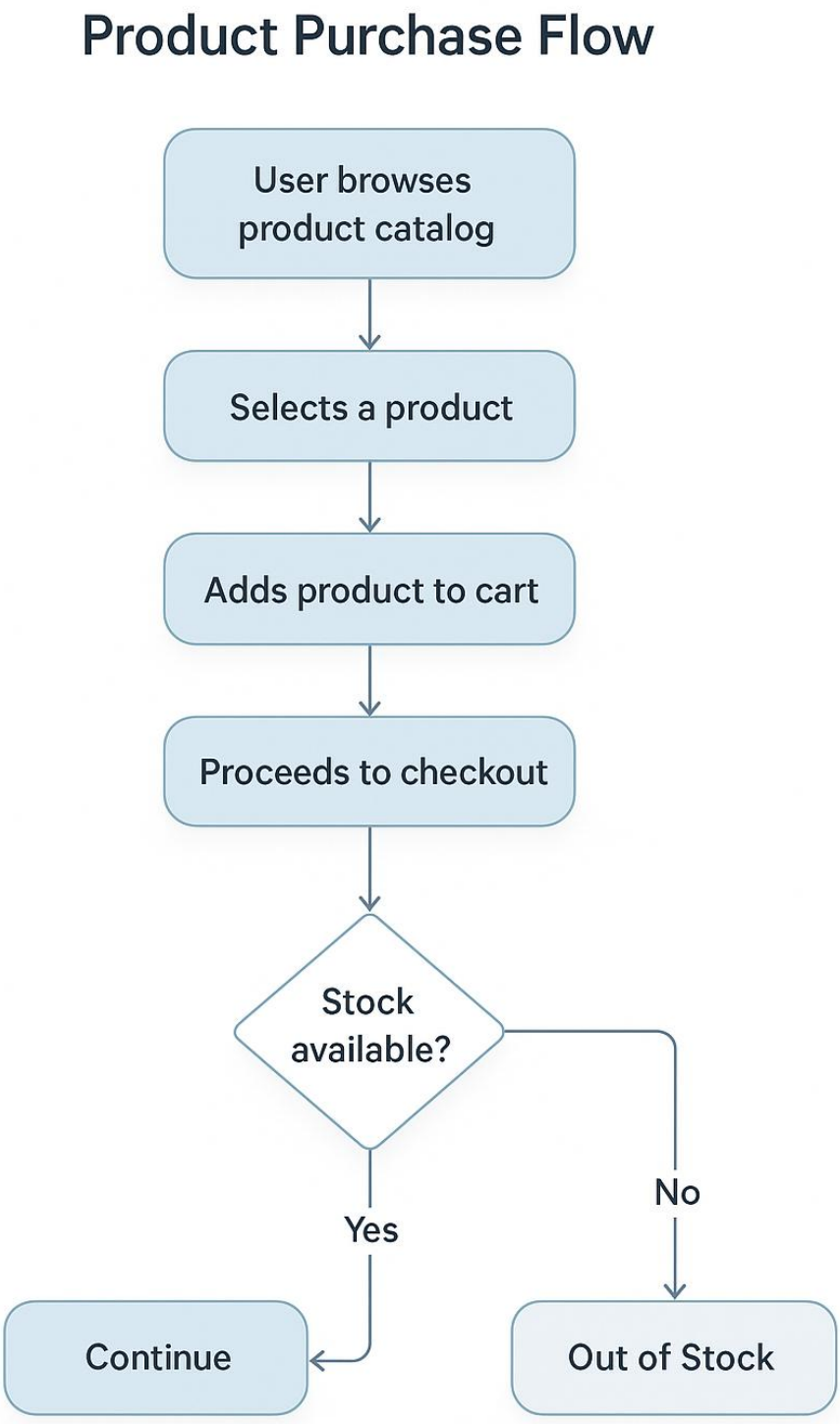


7.2 Product Purchase Flow

Process Steps:

1. User browses product catalog.
2. Selects a product.
3. Adds product to cart.
4. Proceeds to checkout.
5. System verifies stock availability.
6. If available → Continue.
7. If not available → Show “Out of Stock.”

Flow Chart Structure (Textual):

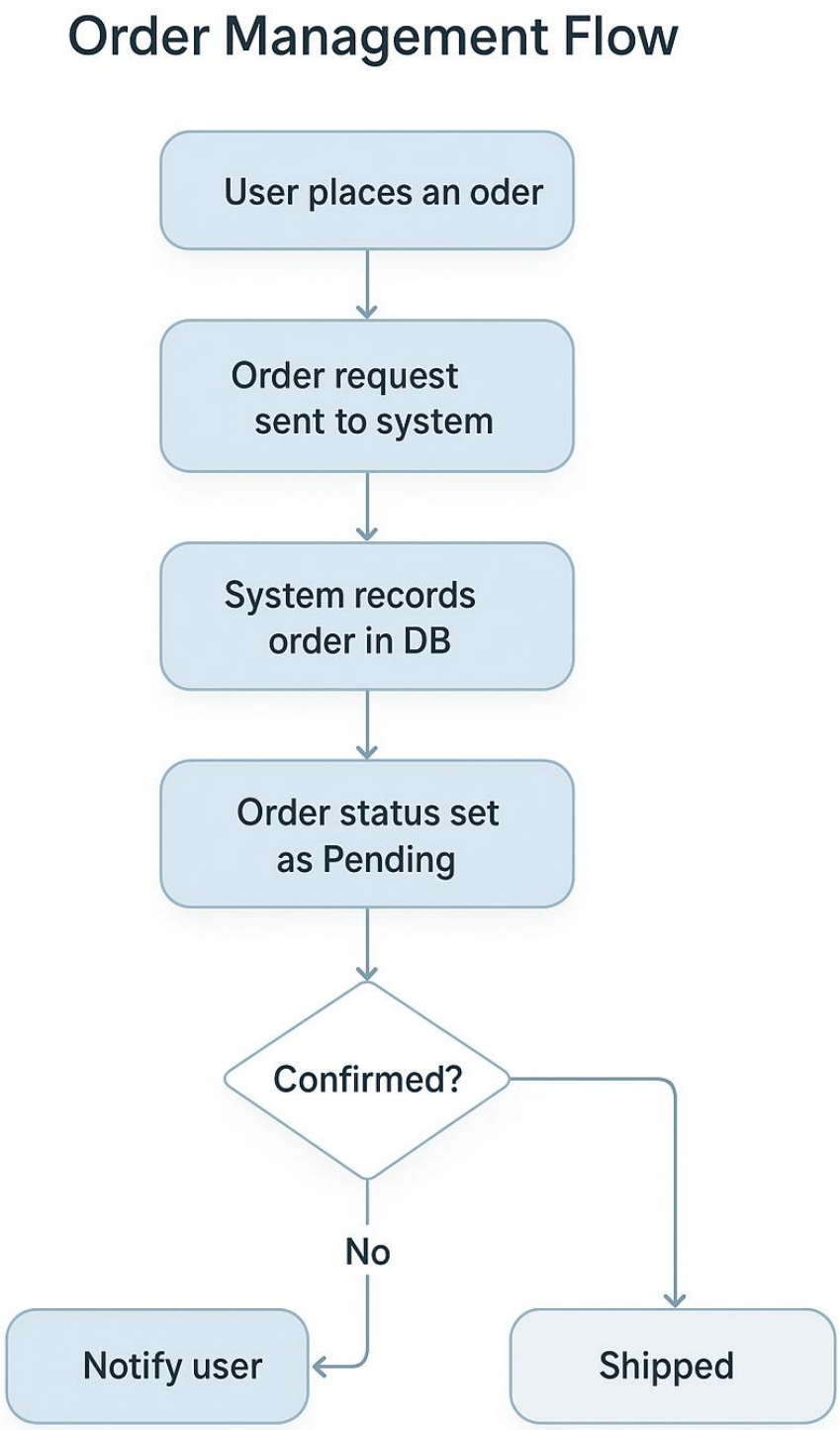


7.3 Order Management Flow

Process Steps:

1. User places an order.
2. Order request sent to system.
3. System records order in DB.
4. Order status set as **Pending**.
5. Admin verifies and confirms order.
6. If confirmed → Move to **Processing**.
7. If cancelled → Notify user.
8. Delivery staff updates status as **Shipped → Delivered**.

Flow Chart Structure (Textual):

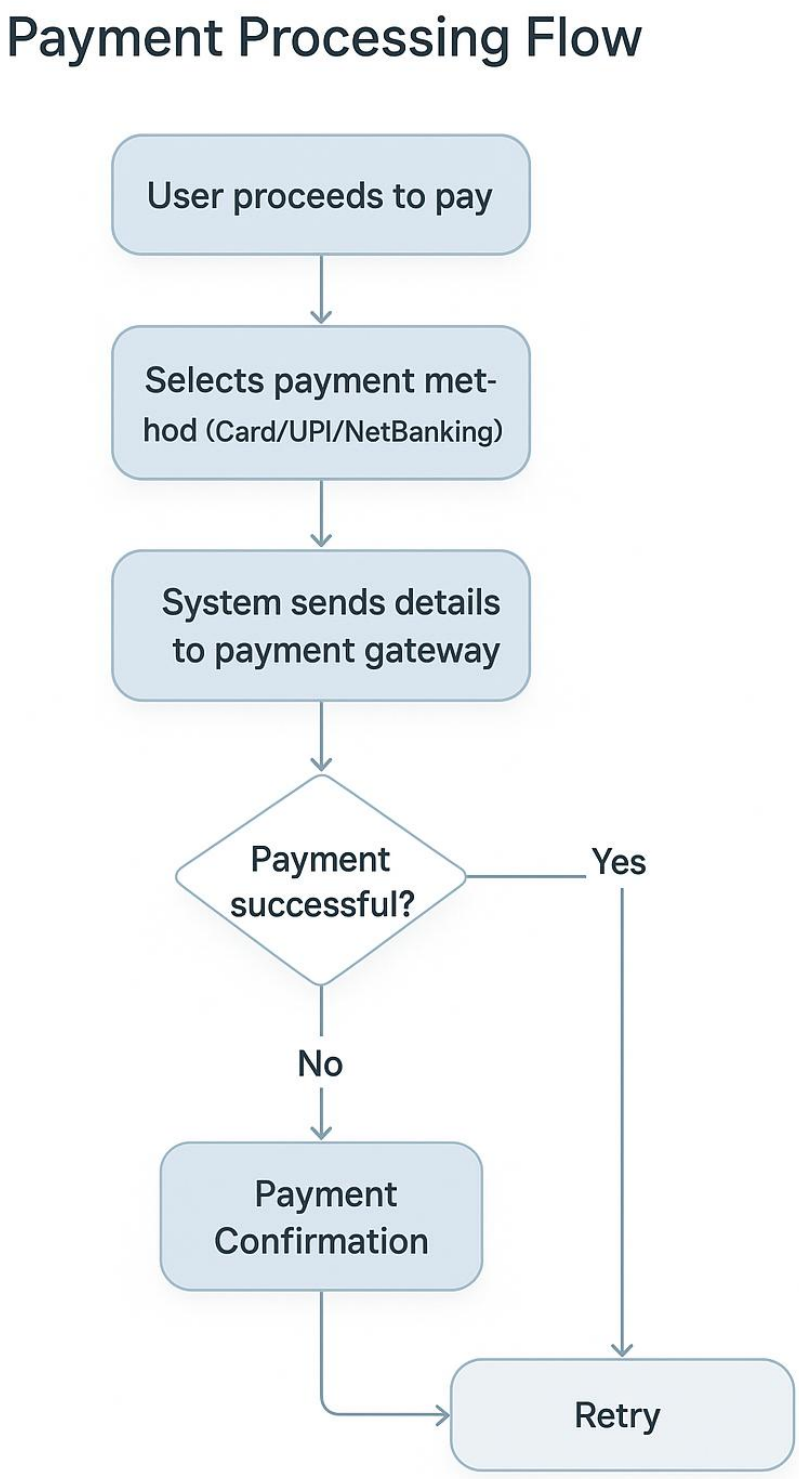


7.4 Payment Processing Flow

Process Steps:

1. User proceeds to payment.
2. Selects payment method (Card/UPI/NetBanking/Cash).
3. System sends details to payment gateway.
4. Gateway verifies payment.
5. If successful → Payment Confirmation → Order Completed.
6. If failed → Error Message → Retry option.

Flow Chart Structure (Textual):



8. Implementation Details

8.1 Frontend (HTML, CSS, JavaScript, Bootstrap, AOS)

- The frontend of the system is developed using **HTML, CSS, and JavaScript**, ensuring a clean, structured, and user-friendly interface.
- **HTML** is used to design the structure of the web pages.
- **CSS** is applied to enhance the visual appearance, providing styling, colors, fonts, and layout management.
- **JavaScript** is utilized for client-side interactivity and dynamic functionalities.
- **Bootstrap 5** is used for responsive layouts, grids, and pre-styled UI components like buttons, modals, and navigation.
- **AOS (Animate On Scroll)** library is integrated to enhance the user experience with smooth animations during scrolling.
- Major pages include:
    - **Home Page** → Displays featured products and categories.
    - **Product Pages (Hero, Honda, Bajaj)** → Brand-wise product listings.
    - **Cart Page** → Displays cart items dynamically.
    - **Checkout Page** → Checkout form with user details and payment selection.

- **Profile, Orders, Delivery Address** → User account management.

- The frontend communicates with backend APIs using **AJAX/Fetch** (to exchange JSON data with the server).

8.2 Backend (PHP – Core PHP for Logic & Server-Side Processing)

- The backend of the system is developed using **Core PHP**, which handles all the server-side logic and database interactions.
- Instead of APIs, PHP pages directly communicate with the **MySQL database** and generate dynamic content for the frontend.
- **Key functionalities implemented in PHP include:**
  - **User Authentication** – Login and Registration modules using PHP sessions for maintaining user state.
  - **Product Management** – Adding, updating, deleting, and displaying products using MySQL queries.
  - **Cart Operations** – Adding items to the cart, updating quantities, and removing products.
  - **Order Processing** – Handling order placement, storing order details, and tracking status.
  - **Payment Handling (Demo/Mock)** – Providing options like Cash on Delivery, Card, and UPI (simulated).

- **Admin Panel** – Managing products, users, and orders from the backend interface.

- **PHP sessions** are used for maintaining login status, cart data, and order flow across different pages.
- All database operations are handled via **MySQLi** queries for secure and efficient interaction.

8.3 Database (MySQL – Tables & Schema)

- The project uses **MySQL** as the backend relational database management system (RDBMS).
- **phpMyAdmin** or MySQL CLI is used to manage the database.
- The schema is designed using multiple interrelated tables with **primary keys, foreign keys, and relationships** to ensure data integrity.
- **Main Tables in the Database:**
  - **Users** → Stores user details (user_id, fullname, email, contact, birthdate, gender, password).
  - **Admins** → Stores administrator login details (admin_id, email, password, created_at).
  - **Products** → Contains product information (product_id, name, brand, category, price, stock, discount, image, status).

- **Cart** → Maintains user's selected items (cart_id, user_id, product_id, quantity).
- **Orders** → Stores confirmed orders (order_id, user_id, customer details, shipping address, total_amount, payment_method, created_at).
- **Order_Items** → Stores products inside each order (order_item_id, order_id, product_id, product_name, quantity, price).
- **Payments** → Stores transaction details (payment_id, order_id, method, amount, status, transaction_date).
- **DeliveryRecords** → Tracks delivery (delivery_id, order_id, staff_name, delivery_address, status, delivered_at).
- **Reports** → Admin analytics reports (report_id, type, generated_at, data).

- **Database Design Features:**
  - Proper use of **Primary Keys** for unique identification.
  - **Foreign Keys** to establish relationships (e.g., order_items linked to orders and products).
  - Use of **constraints** like NOT NULL, UNIQUE, and AUTO_INCREMENT for data consistency.
  - **Normalization** applied to reduce redundancy and improve query efficiency.

8.4 Security Features (Authentication, Session Management, Validation)

- **Authentication**
  - Secure login using hashed passwords (bcrypt/MD5).
  - Session tokens issued on successful login.
- **Session Management**
  - PHP sessions maintain user state across requests.
  - Token-based authentication for React frontend API calls.
- **Validation**
  - Client-side (React) form validation for inputs.
  - Server-side (PHP) validation for email, password, and payment details.
- **Data Security**
  - Passwords stored in encrypted format.
  - APIs restricted with role-based access (User/Admin).
- **Other Features**
  - Prevention of SQL Injection (Prepared Statements).
  - Prevention of Cross-Site Scripting (XSS) using input sanitization.

9. Testing

9.1 Test Cases (Unit & Integration)

Test Case ID	Module	Test Scenario	Input	Expected Output	Result
TC-01	Login & Registration	User enters valid credentials	Email: user@test.com, Password: 1234	Redirect to dashboard/homepage	Pass
TC-02	Login & Registration	User enters invalid credentials	Email: wrong@test.com, Password: xyz	Show error message “Invalid credentials”	Pass
TC-03	Cart Management	Add product to cart	Product ID: 101, Qty: 2	Cart updated with correct product & quantity	Pass
TC-04	Cart Management	Remove product from cart	Remove Product ID: 101	Cart updated with product removed	Pass
TC-05	Order Placement	Place order with valid cart	Cart → Checkout → Confirm	Order ID generated, status = “Pending”	Pass

Test Case ID	Module	Test Scenario	Input	Expected Output	Result
		items			
TC-06	Payment Processing	Valid payment via UPI/Card	Payment details entered	Payment Success message, status = “Paid”	Pass
TC-07	Payment Processing	Invalid payment details	Wrong Card Number	Show error message “Transaction Failed”	Pass
TC-08	Admin Management	Add new product	Product details form	Product stored in DB & visible in product list	Pass
TC-09	Admin Management	Generate Sales Report	Click “Generate Report”	Report file created with correct data	Pass

---

9.2 User Testing (UI/UX & Functional)

User testing was carried out with a group of **10 students and faculty members**. The testing focused on **UI clarity, navigation, responsiveness, and functional workflows**.

- □ **Login/Register:** Smooth and easy to use.
- □ **Product Browsing:** Search and filter features worked correctly.
- □ **Cart & Checkout:** Easy navigation from cart to payment.

- □ **Responsiveness:** Website adapted well to desktop and mobile screens.
- □ **Animations (AOS):** Added better user experience without slowing down performance.

Feedback:

- Minor improvements suggested in **search filtering speed** and **dark mode option**.

9.3 Admin Testing

Admin panel functionalities were tested for **CRUD operations, report generation, and order management**.

- □ Admin successfully added, updated, and deleted products.
- □ Order management system allowed status update (Pending → Shipped → Delivered).
- □ Sales & Inventory Reports generated correctly.
- □ User management (block/unblock) worked as expected.

9.4 Performance & Load Testing

Performance testing was performed using **JMeter & manual load simulation**.

Test Type	Scenario	Result
Page Load Speed	Homepage load under 2 seconds	□ Passed
Concurrent Users	50 users browsing simultaneously	□ Passed

Test Type	Scenario	Result
Order Placement Stress	20 users placing order at same time	□ Passed
Database Query Speed	Fetching 1000+ products with filters	□ < 1.5s
Payment Gateway Response	Average response time < 3s	□ Passed

---

10. Limitations of the Project

10.1 Technical Limitations

- The system is developed using **Core PHP and MySQL**, which may limit scalability when handling a very large number of concurrent users compared to modern frameworks.
- Complex queries in MySQL involving large datasets may affect performance if proper indexing and optimization are not applied.
- Security measures such as advanced encryption, prepared statements for all queries, and modern authentication mechanisms (e.g., OAuth2.0, multi-factor authentication) are not fully implemented, which may limit robustness.
- The system currently relies on a **local server/XAMPP setup** during development, which restricts scalability, performance, and availability compared to cloud-based deployment.

10.2 Functional Limitations

- Payment methods are limited to **basic transactions (Cash, UPI, Net Banking, Debit/Credit Card)** and do not yet support **wallets, EMI, or international gateways**.

- The **delivery management system** is simplified and lacks real-time tracking or integration with third-party logistics providers.
- Reporting and analytics features are basic and do not provide **deep insights or predictive analytics** for sales, user behavior, or inventory trends.
- No dedicated **mobile app** is currently available, restricting accessibility to desktop and web browsers only.

10.3 Future Enhancements

To overcome the above limitations and improve the system, the following enhancements can be considered in future iterations:

- Develop a **mobile application (Android & iOS)** for wider accessibility.
- Implement **AI-driven product recommendations** based on user behavior and purchase history.
- Introduce **advanced reporting and dashboards** for admins, including sales forecasting, inventory optimization, and customer insights.
- Enable integration with **third-party delivery services (e.g., Delhivery, Shiprocket)** to provide real-time tracking.
- Expand **payment gateway support** to include PayPal, Razorpay, wallets, and international payment methods.
- Deploy the system on **cloud platforms (AWS, Azure, or GCP)** for better scalability, security, and performance.

---

11. Conclusion

11.1 Summary of Work

The project **“eAuto – Bike Spare Parts E-Commerce Platform”** was designed and developed to provide a seamless digital marketplace for two-wheeler spare parts.

The system integrates:

- **Frontend:** HTML, CSS, JavaScript, Bootstrap, and AOS for a responsive and interactive user interface.
- **Backend:** Core PHP for server-side logic and business processing.
- **Database:** MySQL for handling product, user, order, and transaction records.

The platform supports **user registration, login, product browsing, cart management, order placement, and payment processing**, along with **admin functionalities** such as managing products, users, inventory, orders, and reports. This project successfully bridges the gap between suppliers, customers, and delivery systems, offering a reliable solution for bike spare parts e-commerce.

11.2 Achievements

- Developed a **fully functional e-commerce system** with **user-side and admin-side modules**.

- Implemented **secure authentication and session management** for protecting user accounts.
- Integrated **cart, checkout, and order management workflows** in real-time.
- Designed a **MongoDB database schema** to handle collections such as users, products, orders, payments, and deliveries.
- Provided **basic reporting and analytics** for admins to monitor sales, inventory, and performance.
- Delivered a **responsive and user-friendly interface** with animations for better UX.

11.3 Future Scope

The project lays the foundation for a scalable and advanced platform. In the future, it can be enhanced with:

- **Mobile applications (Android/iOS)** for better accessibility.
- **AI-driven product recommendations** to personalize the shopping experience.
- **Advanced reporting & dashboards** with predictive analytics for business intelligence.
- **Integration with third-party delivery partners** for real-time shipment tracking.
- **Expanded payment options**, including PayPal, wallets, EMI, and international gateways.

- **Cloud deployment** (AWS, Azure, GCP) for improved scalability, performance, and global reach.

---

12. References

1. **React JS Documentation** – https://react.dev
2. **Bootstrap 5 Documentation** – https://getbootstrap.com
3. **MongoDB Documentation** – https://www.mongodb.com/docs
4. **PHP Official Documentation** – https://www.php.net/docs.php
5. **MDN Web Docs (HTML, CSS, JavaScript)** – https://developer.mozilla.org
6. **W3Schools Tutorials** – https://www.w3schools.com
7. **GeeksforGeeks (Web Development & Database Concepts)** – https://www.geeksforgeeks.org
8. TutorialsPoint – https://www.tutorialspoint.com
9. Stack Overflow – https://stackoverflow.com (for troubleshooting and coding references)
10. Research papers and articles on **E-Commerce Platforms, Payment Systems, and Inventory Management** accessed via Google Scholar.
